

ENGINEERING · ARCHITECTURE

Inventre

High-Level Design

System architecture of the Inventre school-commerce platform: storefront, admin back office, fee ledger, and the integrations that connect them to fulfilment, payments and schools.



DOCUMENT	DATE	STATUS	OWNER
HLD v1.7	23 September 2026	Baseline (as-built)	Inventre Engineering

Companion document: **Inventre — Low-Level Design (LLD) v1.7**. This HLD describes *what* the system is and how its parts fit; the LLD describes *how* each part is built.

Contents

01	Introduction	3
02	System overview	4
03	Architecture	5
04	Technology stack	7
05	Data architecture	8
06	External interfaces & integrations	10
07	Deployment & infrastructure	11
08	Security architecture	13
09	Non-functional requirements	14
10	Assumptions, constraints & risks	15
11	Findings & rectification register	16
12	Glossary & references	19

Revision history

Version	Date	Author	Change
1.7	2026-09-23	Engineering	Readability revision: Inter typeface (the previous font file rendered with collapsed word spacing in PDF), larger type scale, every diagram redrawn at reading size with fewer elements. No content change.
1.6	2026-09-23	Engineering	Backups: midnight IST schedule, code pushed to GitHub on every commit, uncommitted edits and <code>.env.deploy</code> mirrored every 5 min, self-contained recovery kit in the backup folder; F-04 updated (Microsoft app secret rotated).
1.5	2026-09-23	Engineering	F-11 closed: point-in-time database backup, code bundles and encrypted off-host mirror to Microsoft 365 (commits <code>e6e7c07</code> ... <code>e594b6f</code>); backup section rewritten.
1.4	2026-09-23	Engineering	F-08 closed (storefront pages split, smoke + axe script), F-15 code items closed (commit <code>cf0ca1d</code>); F-04, F-11 and the palette-contrast note re-stated as owner decisions.
1.3	2026-09-23	Engineering	Medium and low findings F-05 ... F-22 rectified and deployed (commits <code>c538354</code> , <code>6e9f4f5</code>); body text updated to the as-deployed state; F-08 and F-11 re-scoped.
1.2	2026-09-23	Engineering	F-04 credential rotation: local secrets rotated and deployed (commits <code>b3d375c</code> ... <code>96eb4c9</code>); DB password scripted; shared-secret list; F-22 / F-23 added.
1.1	2026-09-23	Engineering	F-01, F-03 and F-17 rectified and deployed to production (commits <code>934aab7</code> , <code>bff65e6</code>); register updated; F-21 added.
1.0	2026-09-23	Engineering	Initial as-built baseline generated from the production repository (branch <code>feat/ground-stock-availability</code> , 145 commits, last commit 2026-09-16).

01 Introduction

1.1 Purpose

This High-Level Design (HLD) records the architecture of Inventre as it runs in production on 23 September 2026. It is the reference for onboarding engineers, for change impact assessment, for audits, and for planning the next phase of the platform. It is written from the source code and the live deployment, not from earlier plans, so where a plan and the running system disagree, this document follows the running system.

1.2 Scope

IN SCOPE

- Inventre storefront (inventre.in) and its public pages
- Inventre admin back office (/admin)
- Fee ledger console (/fees) and its MyClassBoard import
- Parent concern portal (/portal) and support view-as-parent
- Backend API, database, cache, media storage, background jobs
- Integrations with the Audit ERP, ERPNext, CCAvenue, SMS gateway, e-mail, MyClassBoard
- Deployment topology on the production host and the dev/test environments

OUT OF SCOPE

- Internal design of the Audit ERP (separate FastAPI system, separate repository and host)
- Employee Rewards Marketplace and Task Manager (separate applications that share the host only)
- Procurement application (third-party system)
- Business processes at schools and the warehouse

1.3 Audience

Engineering, DevOps, QA, product management and external auditors. Section 3 to 7 assume familiarity with web architecture; Section 2 is readable by non-engineers.

1.4 Conventions

- Paths such as `app/api/cart` are relative to the repository root `/root/Inventre`.
- "Audit ERP" means the in-house fulfilment ERP at `audit.inventre.onLine`; "ERPNext" means the legacy ERP at `erp.inventre.in`.
- Money values inside the system are stored in paise (integer); the UI renders rupees.
- Status pills: Live in production, Partial built but limited, Gap missing or defective, Planned.

READING GUIDE

Section 11 lists every drawback found while producing this document, with an owner-ready rectification for each. Items marked Gap in the body are cross-referenced there by ID (F-01 ... F-23).

02 System overview

Inventre is an e-commerce platform for school uniforms, books and kits. A parent signs in with their mobile number, sees only the catalogue for their child's school and grade, builds an order (often a pre-configured "Magic Box" kit), and pays through CCAvenue. Inventre's own Postgres database is the system of record for the storefront; a bridge keeps the fulfilment ERP in sync and pulls delivery status back.

2.1 Actors

Actor	Interacts through	Needs
Parent / guardian	Storefront /shop , /account , public /portal	Log in by phone OTP or password, see the right catalogue for each child, order, pay, track, request exchanges or missing items, raise concerns.
Inventre operations staff	Admin /admin (roles super , ops)	Manage catalogue, schools, students, orders, stock, pricing, returns, content, users and integrations; export Excel reports.
School administrator	Admin /admin (role school_admin , read-only)	See their school's students and orders.
Fee desk / management	Fee console /fees (roles fees-viewer , fees-admin)	View MyClassBoard fee ledger, receivables ageing, receipts; manage fee-desk users.
Support agent (Audit ERP call centre)	Support view handoff (read-only "view as parent")	See exactly what a parent sees without an OTP; never mutate.
Audit ERP (system)	HTTPS + HMAC ingest, JWT-authenticated reads, webhooks	Receive orders, students, guardians, exchange and missing requests, concerns; return statuses.
CCAvenue (system)	Hosted payment page, encrypted callback, Status API, SFTP settlement files	Collect payment, report result.
MyClassBoard (system)	REST API, nightly import	Provide student roster, fee schedule and receipts per branch.

2.2 Business capabilities

COMMERCE - School- and grade-scoped catalogue - Products, variants (size / colour), bundles, Magic Box configurator, bookkits with language selection - Per-school pricing, discounts, coupons, delivery fee rules, GST - Cart per sibling, CCAvenue checkout, offline orders	AFTER-SALES - Order tracking with per-item status - Exchange and missing-item requests (7-day window after last delivery) - Parent concern portal with ticket tracking - Support view-as-parent for call-centre agents	OPERATIONS - Admin back office (~125 pages) - Students, guardians, schools, grades, bulk import - Stock (bins, ledger, Ground Stock bridge), purchase orders, suppliers - Reports, invoices, GST returns data, activity log - Fee ledger and receivables ageing (MyClassBoard)
--	---	---

2.3 Key figures (production repository, 2026-09-23)

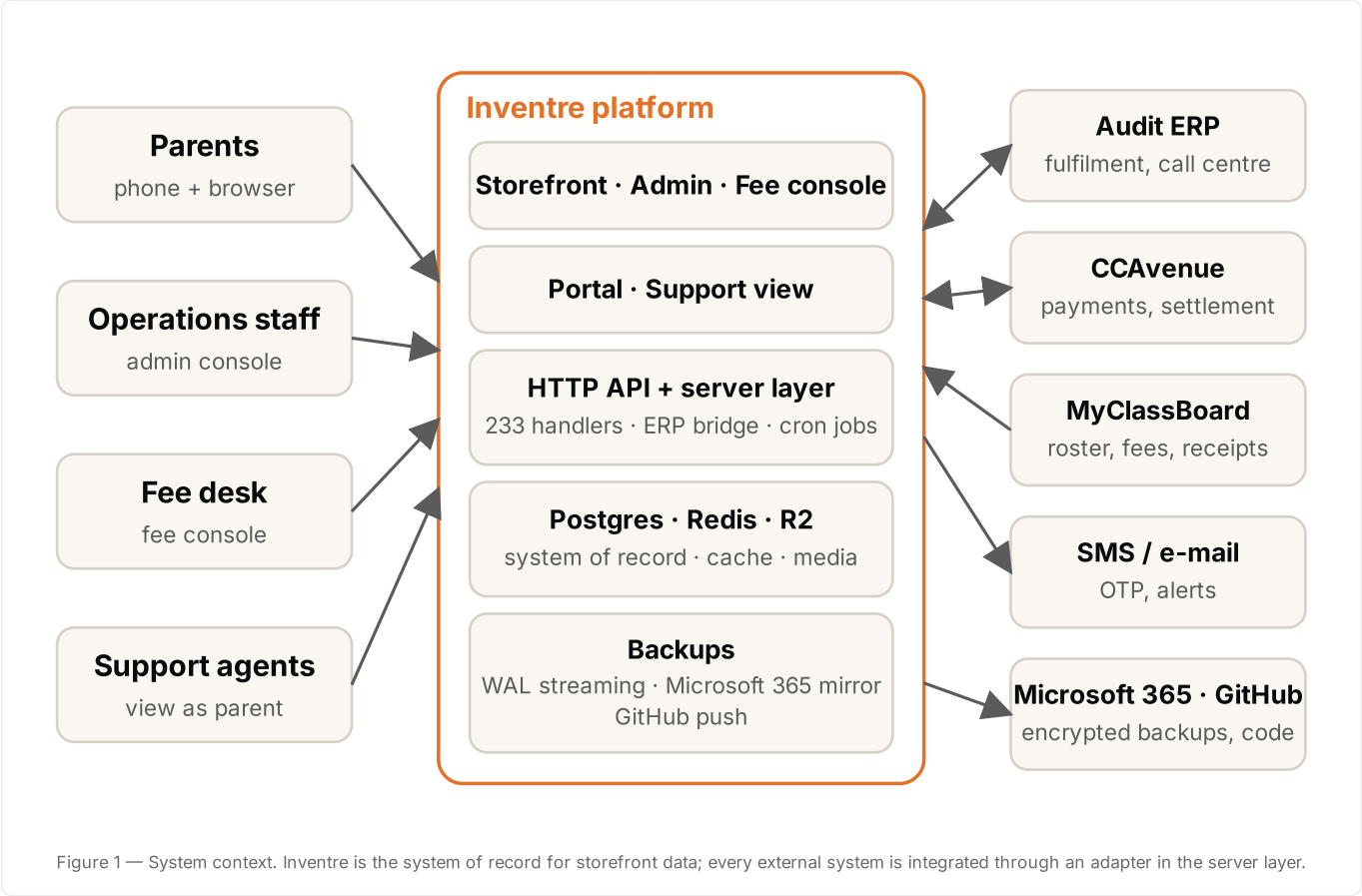
Measure	Value	Measure	Value
TypeScript / TSX / SQL lines	≈174,400	Database tables	96 in Drizzle schema + 12 created by raw SQL migrations
API route handlers	233	SQL migrations	92 files (0000 → 0079 with sub-steps)
Pages (non-API)	152	Server-side modules (server/)	122 files
React components	176	Operational scripts (scripts/)	191
Cron endpoints	12	Automated tests	10 Vitest files (unit, domain, architecture invariants, regression)

03 Architecture

3.1 Architectural style

Inventre is a **modular monolith**: one Next.js 15 application serves the storefront, the admin console, the fee console, the public portal and the entire HTTP API from a single deployable container. Business logic lives in a server-only layer (`server/`) that both the API route handlers and the server-rendered pages call. Long-running or scheduled work is not a separate worker process; it is exposed as authenticated `/api/cron/*` endpoints that the host's cron daemon calls. This keeps deployment simple (one image, one container) at the cost of coupling the job runtime to the web runtime.

3.2 System context

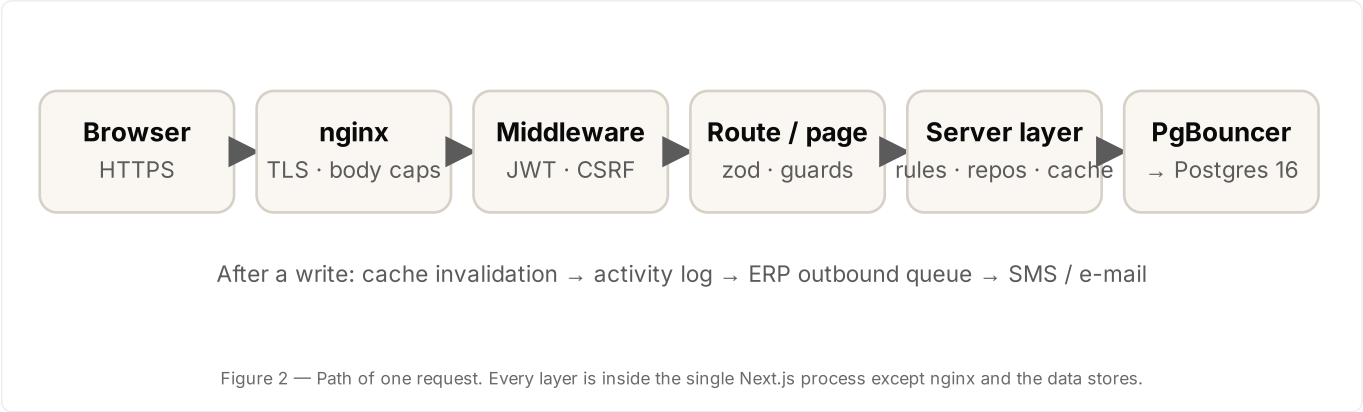


3.3 Logical components

Component	Location	Responsibility
Storefront UI	<code>app/shop</code> , <code>app/account</code> , <code>app/login</code> , <code>components/shop</code>	Client-rendered React pages that call the HTTP API. Cart context, Magic Box configurator, PDP, checkout, orders, exchange/missing forms.
Marketing site	<code>app/page.tsx</code> , <code>/about</code> , <code>/contact</code> , <code>/experience-store</code>	Server-rendered brand pages, cached at the edge for 60 s / 600 s.
Admin console	<code>app/admin/(protected)</code> , <code>components/admin</code>	~121 server-rendered pages behind a layout guard and per-section permission gates. Own primitive kit (<code>components/admin/ui</code>).
Fee console	<code>app/fees</code>	Standalone MyClassBoard ledger UI with its own session cookie, fonts and CSS namespace.
Public portal & support view	<code>app/portal</code> , <code>app/support-view*</code>	No-login concern intake; read-only impersonation for agents.
HTTP API	<code>app/api/**</code> (233 handlers)	Route handlers validated with zod via <code>server/api-handler.ts</code> ; grouped by audience: parent, admin, fees, cron, erp (inbound), portal, support.
Domain / server layer	<code>server/**</code> (122 files)	Sessions, guards, repos (data access), bundle engine, pricing, tax, eligibility rules, numbering, notifications, ERP bridge, payment finaliser, importers, jobs.
Data layer	<code>db/schema.ts</code> , <code>db/migrations</code> , <code>db/client.ts</code>	Drizzle ORM schema, SQL migrations, postgres.js client through PgBouncer.

Component	Location	Responsibility
Integration adapters	<code>server/erp*.ts</code> , <code>server/erp/</code> , <code>server/ccavenue*.ts</code> , <code>server/notify</code> , <code>server/mcb</code> , <code>server/storage.ts</code> , <code>lib/mcb</code>	One adapter per external system; outbound queue with retries and dead-letter for the ERP.
Scheduled jobs	<code>app/api/cron/*</code> + <code>/etc/cron.d/inventre-*</code>	ERP drain/poll/reconcile, CCAvenue reconcile, settlement, webhooks retry, cart cleanup, auto-PO, Ground Stock sync, MCB imports, DB backup, monitor.
Edge middleware	<code>middleware.ts</code>	JWT signature check for the four session cookies, CSRF origin check, read-only enforcement for support view, redirect to the right login.

3.4 Request flow



3.5 Rendering model

The two halves of the application render differently, and that is intentional but worth knowing:

- **Storefront pages are client components.** `/shop` , the product page, cart, checkout and orders are marked `"use client"` and fetch JSON from `/api/*` with `cache: "no-store"` . A focus-refetch hook refreshes data when the tab regains focus. Exception: the exchange and missing pages are server components that read the database directly.
- **Admin pages are server components.** 104 of 122 protected pages import the database or server layer directly and render on the server; writes go through `/api/admin/*` (only four server actions exist).
- **Everything is dynamic.** About 115 pages export `dynamic = "force-dynamic"` ; there is no ISR or tag revalidation. Cache-Control headers make `/shop/*` `private, no-store` and the home page publicly cacheable for 60 s.

04 Technology stack

Layer	Technology (version)	Role and notes
Runtime	Node.js 20 (Alpine), TypeScript 5.6	Single process; standalone Next.js output in a ~50 MB image.
Web framework	Next.js 15 (App Router), React 18.3	Pages, layouts, route handlers, middleware, instrumentation hook.
Styling	Tailwind CSS 3.4, PostCSS, clsx + tailwind-merge	Design tokens <code>brand</code> (#E47127), <code>ink</code> , <code>cream</code> ; Plus Jakarta Sans via next/font. Fee console uses Archivo + IBM Plex Mono and a scoped stylesheet.
UI libraries	lucide-react, framer-motion 11	Icons in 238 files; motion in marketing, auth and shop.
Validation	zod 3	All API bodies and query strings; a shared <code>parseJson</code> / <code>parseQuery</code> / <code>apiHandler</code> .
ORM / DB	Drizzle ORM 0.36, drizzle-kit 0.30, postgres.js 3.4, PostgreSQL 16	Typed schema in <code>db/schema.ts</code> ; SQL migrations run by a custom journal-independent migrator on container boot.
Pooling	PgBouncer (edoburu image), transaction mode	App pool 30 → PgBouncer pool 50 → Postgres max 100.
Cache / ephemeral	Redis 7 (ioredis 5)	OTP codes, rate-limit counters, catalogue cache with pattern invalidation. Password-protected since 2026-09-10.
Object storage	Cloudflare R2 (prod), MinIO (local); AWS SDK v3 S3 client	Product and ERP media; <code>/images/*</code> redirect to R2 <code>/v2/</code> keys.
Auth	jose (JWT HS256), @node-rs/bcrypt / bcryptjs	Four httpOnly cookies; passwords hashed with bcrypt.
Messaging	Arihant Global SMS API (OTP), MSG91 (templates configured), nodemailer SMTP, Resend key present	OTP is the only SMS that actually sends today (see F-09).
Payments	CCAvenue (hosted page, AES-encrypted callback, Status API, SFTP settlement .xlsx)	Finaliser is idempotent across callback, poll, cron and settlement.
Spreadsheets	xlsx 0.18	Admin exports (students, products, price lists, fees ageing).
Testing	Vitest 2, Playwright 1.60 (scripts only)	Pure-function unit tests, architecture invariants, row-count regression.
Quality gates	ESLint 9 flat config, Prettier 3, tsc strict check, GitHub Actions CI, deploy preflight	Deploy refuses a dirty tree or failing typecheck/tests.
Infrastructure	Docker Compose, nginx + certbot, cron, Prometheus + Grafana + blackbox (deploy/monitoring)	Single production host; dev and test stacks side by side.

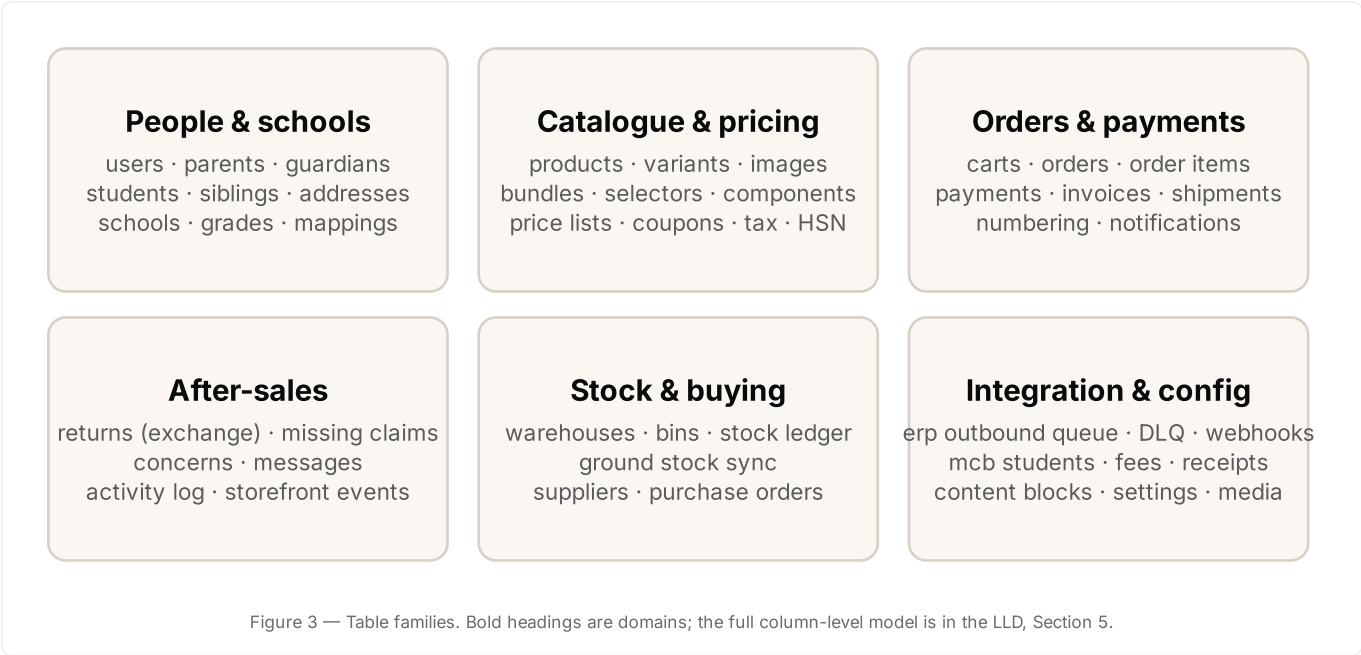
PLAN VS REALITY

The original architecture plan (`docs/architecture/PLAN.md`) named Better Auth, MSG91, Razorpay, BullMQ, Caddy and Coolify. None of these are in the running system: auth is hand-rolled JWT, payments are CCAvenue, jobs are cron-driven HTTP endpoints, and the proxy is nginx. The plans were moved to `docs/history/` on 2026-09-23 and the README now points here (F-14).

05 Data architecture

One PostgreSQL 16 database (`inventre`) is the system of record. Redis holds only ephemeral data. Media binaries live in Cloudflare R2 with metadata rows in Postgres.

5.1 Domain map



5.2 Principles in force

- **Money in paise.** All amounts are integer paise; exports label units explicitly.
- **Snapshots at order time.** Order items snapshot name, price, tax, size and colour so later catalogue edits do not rewrite history.
- **Soft state through enums.** 34 Postgres enums (`order_status`, `payment_status`, `return_status`, ...) constrain lifecycle columns.
- **Canonical grade.** `students.grade` is the single source for a student's grade; the ERP receives it verbatim (see LLD 5.4 for the +3 legacy offset trap).
- **Append-only trails.** `activity_log` (admin actions), `stock_ledger`, `otp_logs`, `storefront_events` are never updated in place.
- **Migrations are the only schema path.** 92 SQL files; the container runs the migrator before `next start`. Verified on 2026-09-10 to rebuild an empty database to 94 tables / 1,100 columns.

5.3 Caching and ephemeral state

Store	What	Lifetime
Redis	OTP codes and attempts, rate-limit buckets, catalogue read cache (<code>server/cache.ts</code> with <code>invalidateCatalog()</code>), cart mirror for guests	Seconds to 7 days; <code>allkeys-lru</code> , 512 MB
Browser localStorage	Active student, order-status "seen" markers, PDP drafts for guests	Per device; never authoritative
Postgres <code>carts</code>	Authoritative cart per parent + student	Cleaned every 6 h by <code>cleanup-carts</code>

5.4 Backup and recovery

- **Point-in-time:** WAL streamed continuously to `/root/backups-archive/wal` (systemd `inventre-wal-stream`, slot `inventre_wal`) + full base backup 00:00 IST → restore to any second within 30 days.
- **Logical:** nightly dump 00:10 IST (`scripts/backup-db.sh`, 14 daily + monthly); 6-hourly snapshot (dump, AES-256 env, MinIO volume); `.env.deploy` also mirrored on its own, re-encrypted within 5 min of a change.
- **Code:** on every commit (git hook), after every deploy and at 00:20: bundle of all branches to the mirror and push to GitHub `Sunith0801/Inventre-Website`; uncommitted edits packed every 5 min.
- **Off-host, encrypted:** rclone every 5 min to Microsoft 365 (SharePoint) through a crypt layer; failure and staleness e-mails; weekly automatic restore test with e-mail. **Recovery kit** in the same folder: `RESTORE/bootstrap-new-server.sh` rebuilds a fresh server from the folder plus one passphrase (`docs/runbooks/restore-from-m365.md`); manual fallback `docs/runbooks/host-rebuild.md`.

- Restore of a production dump was verified on 2026-09-10 (refactor baseline).
- A streaming relay replica design exists (`staging.inventre.in` writable clone via `inventre-relay-postgres`) and a Redis Sentinel guide is in `deploy/replicas` .
- Live Off-host copies land in the company Microsoft 365 tenant every 5 minutes, encrypted before upload (F-11, 2026-09-23).

06 External interfaces & integrations

System	Direction	Mechanism	Purpose & status
Audit ERP 217.216.58.218 · audit.inventre.online	Outbound push Inbound pull + webhooks	HTTPS POST to <code>/api/ecom/ingest</code> with HMAC signature and per-order event sequence; JWT GETs for order headers, shipments, packing units, customers, students; inbound webhooks for exchange, missing and concern status	Live Orders, students, guardians, addresses, payments, exchange (RTN), missing (MIS), concerns (CON) flow out within 30 s via the drain; delivery status, RTO and per-item packing state flow back every 30 s via the poll. Stale sequence ⇒ HTTP 409, deliberately swallowed.
Audit Ground Stock	Inbound pull	<code>/api/cron/ground-stock-sync</code> every 5 min reads counted stock per SKU and writes bins; a gate switch in the admin Stock report turns availability on the storefront	Live since 2026-09-16. Uncounted or negative SKUs show as sold out by design.
ERPNext (legacy) erp.inventre.in	Inbound pull	Item feed (<code>server/erp/items-feed.ts</code>), education sync (students), media re-hosting to R2; API key + secret	Partial Host firewall blocks the production box from reaching ERPNext; the item feed runs from elsewhere. Kept for catalogue history and BOM references.
CCAvenue	Outbound redirect Inbound callback Outbound status / SFTP	Encrypted request to hosted page; AES-decrypted callback at <code>/api/checkout/ccavenue/callback</code> (CSRF-exempt); Status API poll from the order page and a 5-min cron; hourly settlement .xlsx drained from a chrooted SFTP drop (port 2222)	Live Idempotent finaliser: stock decrement, cart clear, invoice, SMS, ERP enqueue happen exactly once whichever path wins.
MyClassBoard (MCB)	Inbound pull	Token-based REST; nightly import 01:30 (students + fee schedule for 13 branches) and 02:30 (receipts via per-student transactions endpoint)	Live Feeds <code>/fees</code> ledger, ageing and receipts. MCB exposes no receipt PDF and no board field; school is resolved per student by enrolment.
SMS gateway	Outbound	Arihant Global HTTP API for OTP (<code>server/notify/sms.ts</code>); MSG91 template IDs configured	Partial Only OTP sends; transactional messages (order confirmed, exchange received) no-op (F-09). "Insufficient balance" outages are a recharge issue, logged in <code>otp_logs</code> .
E-mail	Outbound	nodemailer over SMTP; Resend key present but unused	Partial Used for alerts (stranded ERP events) and admin notifications.
Cloudflare R2	Outbound	S3 SDK; public dev subdomain for reads; <code>/images/*</code> 307-redirected to <code>/v2/</code> keys	Live Sole media origin in production. MinIO container exists but is unused in prod.
Microsoft 365 (SharePoint)	Outbound	rclone, app-only Entra registration <i>inventre-backup-agent</i> (client credentials, secret <i>Inventre-prod</i>), crypt layer on top	Live Encrypted backup mirror every 5 minutes + recovery kit (F-11).
GitHub	Outbound	SSH deploy key (write) to <code>Sunith0801/Inventre-Website</code>	Live Every branch and tag pushed on each commit, deploy and daily (F-11).
Procurement app 185.193.19.217	—	No direct link from Inventre	Integrated with the Rewards application only. Listed to prevent confusion.

INTEGRATION PRINCIPLE

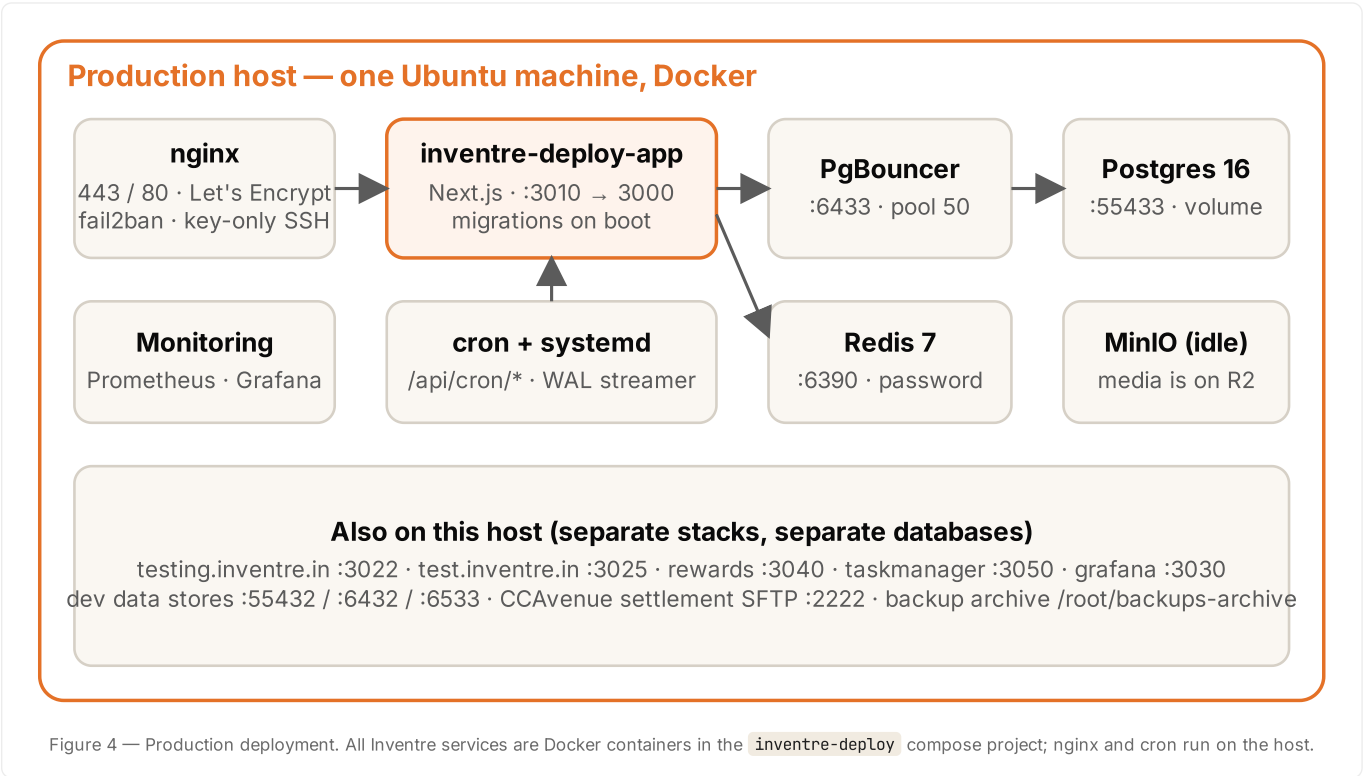
Inventre never calls an external system inside a customer request except CCAvenue redirect and the Status API poll. Everything else is queued or scheduled, so a slow or down partner degrades sync freshness rather than the storefront.

07 Deployment & infrastructure

7.1 Environments

Environment	URL	App port	Database	Source checkout	Notes
Production	inventre.in, www	3010 → container :3000	inventre-deploy-postgres :55433 via PgBouncer :6433	/root/Inventre	Docker image inventre-app:latest ; rollback tags kept per deploy
Testing (dev build)	testing.inventre.in	3022 (build) / 3021 (hot reload)	inventre-dev-postgres :6533	/root/Inventre-dev	Integration branch testing
UI staging	test.inventre.in	3025 (pm2 hot reload)	Own DB inventre_uistaging	/root/Inventre-uistaging	Design review of uncommitted storefront work; deploy script disabled there
Local	localhost:3000	3000	Compose Postgres :55432, PgBouncer :6432, Redis :6379, MinIO :9000	developer laptop	docker compose up -d mirrors prod topology

7.2 Production topology



7.3 Build and release

- 1. Preflight** (`scripts/deploy.sh`): refuses uncommitted source changes unless `--allow-dirty`; runs `npm run typecheck` and `npm test` unless `--skip-verify`.
- 2. Build** on the host with a warm `.next/cache` (~90 s for code-only changes; 3–4 min cold). The Dockerfile is a three-stage build (deps → builder → runner) producing a standalone bundle plus a compiled `db/migrate.js`.
- 3. Release:** `--fast` copies the build into the live container and restarts; full mode recreates the container with `docker compose -p inventre-deploy --env-file .env.deploy up -d --no-deps app`. The previous image is tagged `inventre-app:rollback-*` for one-command rollback.
- 4. Boot:** the container runs the migrator, then `next start`; Docker healthcheck hits `/api/health` (DB + Redis ping).
- 5. Verify:** `verify-deployment.sh` asserts the live `/api/version` reports the build id *and the commit* just built (`gitSha`, `gitBranch`, `dirty`, `builtAt` are baked into the bundle by `next.config.mjs`), all six security headers, and the admin and shop doors; a failure rolls back to the snapshot automatically.
- 6. Post-deploy:** open admin and fee tabs detect the new `BUILD_ID` through `/api/version` and reload themselves (or prompt when a form is dirty).

DEPLOYMENT TRAP — BUILDS FROM THE WORKING TREE (F-01, CLOSED 2026-09-23)

The build context is the working directory of `/root/Inventre`, not a git revision, and on 2026-09-10 a deploy from another checkout replaced production twice. Three layers now close this: every other checkout has its compose file renamed and a refusal guard in `deploy.sh`; every build carries its commit, branch and dirty flag in `/api/version` and `verify-deployment.sh` fails a deploy whose live commit is not the one just built; the 10-minute monitor logs a foreign commit (one not in this checkout's history) as well as a lost security posture.

7.4 Scheduled jobs (production)

Cadence	Endpoint / script	Purpose
All cron lines live in one file, <code>/etc/cron.d/inventre-erp</code> , source <code>.env.deploy</code> and send <code>Authorization: Bearer \$CRON_TOKEN</code> ; no secret is written into cron files (F-03). Every job runs under a Redis lock, so overlapping ticks or a second app instance cannot double-run it (F-07).		
every 30 s	<code>POST /api/cron/erp-drain</code>	Push pending <code>erp_outbound_queue</code> events to the Audit ERP with retry and DLQ
every 30 s	<code>POST /api/cron/erp-poll</code>	Pull order headers, shipments, packing units, RTO into mirror tables
every 2 min	<code>GET /api/cron/retry-webhooks</code>	Re-deliver failed outbound webhook deliveries
every 5 min	<code>GET /api/cron/ccavenue-reconcile</code>	Status API check for pending payments; heals captured-but-stuck orders
every 5 min	<code>POST /api/cron/ground-stock-sync</code>	Audit Ground Stock → bins → storefront availability
every 10 min	<code>POST /api/cron/erp-reconcile</code> · <code>monitor-production.sh</code>	Cross-check local vs ERP order state · detect reverted deploy
every 5 min	<code>scripts/backup/sync-to-m365.sh</code>	Encrypted mirror of all backup artefacts to Microsoft 365
continuous	<code>systemd inventre-wal-stream</code>	Postgres WAL streaming for point-in-time recovery
00:00 · 00:10 · 00:20 · Sun 03:30	<code>base-backup.sh</code> · <code>backup-db.sh</code> · <code>code-bundle.sh</code> · <code>verify-restore.sh</code>	Full DB backup · logical dump · code bundle + GitHub push · weekly restore test (all IST)
hourly :17	<code>GET /api/cron/ccavenue-settlement-reconcile</code>	Drain SFTP settlement .xlsx, mark settled
every 6 h	<code>GET /api/cron/cleanup-carts</code> · <code>snapshot-and-rotate.sh</code>	Purge stale carts · DB snapshot rotation
01:30 / 02:30	<code>cron-import-mcb.sh</code> · <code>cron-import-mcb-receipts.sh</code>	MCB roster + fees · MCB receipts
02:00	<code>GET /api/cron/auto-po</code>	Draft purchase orders for low-stock items
every 1 min	<code>GET /api/cron/erp-webhook-drain</code>	Process stored inbound ERP webhooks (exchange / missing / concern status)
hourly :05	<code>POST /api/cron/erp-stranded-alert</code>	E-mail ops when a paid order has not reached the ERP within the grace window
manual	<code>sync-items</code>	ERPNext item feed; the host cannot reach ERPNext, so it is run from the admin ERP page when needed (decision recorded, F-06)

08 Security architecture

8.1 Identity and sessions

Population	Cookie	Issued by	Lifetime / notes
Parents	inv_session	OTP (Arihant) or password login at /api/auth/*	JWT HS256 signed with JWT_SECRET (jose); httpOnly, Secure, SameSite=Lax. Logging out of /shop clears it (AuthGate).
Admin staff	inv_admin	Email + bcrypt password at /admin/login	7-day TTL; roles super / ops / school_admin plus per-user and per-role permission keys (<page>.read/.write).
Fee desk	inv_fees	/fees/login	Session kind "fees" ; admin guards reject it outright so the ~60 unguarded admin pages are unreachable from a fee account.
Support agents	inv_support_view	Audit ERP mints a token via POST /api/support/mint-view (service bearer), agent opens the handoff URL	Read-only: middleware returns 403 for every mutating method; banner with countdown; explicit exit route.

8.2 Defence layers

EDGE AND TRANSPORT

- TLS via Let's Encrypt; HSTS, X-Frame-Options DENY, Permissions-Policy, enforcing CSP for frame-ancestors / object-src / base-uri , full CSP in report-only
- nginx body caps (12 MB; 60 MB for return photo upload); middleware excludes multipart routes from the 10 MB edge body limit
- Key-only SSH, fail2ban, firewall on Docker-published ports (firewall-docker-ports.sh)

APPLICATION

- Middleware verifies JWT signatures at the edge before any handler runs
- CSRF: SameSite=Lax cookies plus Origin/Referer same-host check on all state-changing JSON APIs (webhooks and the CCAvenue callback are excluded and use signatures / AES instead)
- zod validation on every body and query; apiHandler converts exceptions to JSON, never HTML stack traces
- Redis-backed rate limiting on OTP and login
- All 12 cron endpoints authenticate through one helper, server/cron-auth.ts (Authorization: Bearer CRON_TOKEN , constant-time compare); an architecture test fails the build if a route bypasses it

DATA

- bcrypt password hashes; OTP codes only in Redis with TTL
- Order detail and exchange pages scoped to the requesting family graph (co-guardians 404 outside it)
- Staff names stripped from tracking timelines server-side
- Redis password-protected (2026-09-10); Postgres reachable only on the host

AUDIT AND DETECTION

- activity_log records every admin mutation with before/after
- otp_logs , storefront_events (5xx), communications
- Static analysis run stored in docs/security/SAST-20260909
- Production monitor detects header/authorization regressions every 10 min; gsocket and ssh-it watchdogs after the June 2026 compromise

8.3 Known security gaps

- **Fixed** Admin section gating is now complete. All 39 sections under app/admin/(protected) carry a SectionGate layout, 141 admin API routes call requirePermission , and tests/architecture/invariants.test.ts fails the build if either regresses (verified 2026-09-23; F-02).
- **Fixed** Cron authentication unified on 2026-09-23: one token, one header, constant-time compare, token rotated (F-17).
- **Fixed** Secrets consolidated on 2026-09-23: cron files, the compose file, the deploy script and the MCB import scripts no longer contain any credential; everything is read from .env.deploy (mode 640, owner root, read ACL for the backup user). Stray .env.*.bak copies moved to a root-only archive (F-03).
- **Partial** Credential rotation after the June–July 2026 root compromise: every secret that lives only on this host was rotated on 2026-09-23 (JWT with a 7-day dual-key grace window, AUTH, support-view, Redis, MinIO, Grafana, OTP bypass, cron token). The Postgres password is rotated by scripts/rotate-db-password.sh ; secrets shared with partners still need a coordinated change (F-04).
- **Partial** /api/downloads in the Rewards application grants any private file to any entity admin; not Inventre, but shares the host (tracked there).

09 Non-functional requirements

Quality	Target	How the design meets it / evidence
Performance	Storefront page < 1 s TTFB on 4G; admin lists < 2 s	PgBouncer transaction pooling; Redis read cache for catalogue; home page edge-cached 60 s; Brotli at nginx; standalone Node bundle. Partial Images are plain <code></code> from R2 (decision recorded, F-12); no bundle budget yet.
Scalability	Original plan: 20,000 concurrent users on one host	Stateless app container (sessions in JWT, cart in Postgres/Redis) so a second app replica behind nginx is possible; PgBouncer sized 50 for multi-worker. Ready Every cron endpoint takes a Redis lock (F-07), so a second replica can be added behind nginx without double-running jobs.
Availability	Business hours 99.5 %; nightly windows acceptable	Docker <code>restart: unless-stopped</code> ; healthcheck on DB + Redis; rollback image tags; maintenance page with nginx 503 gate and bypass token. Single host, no failover (F-10).
Reliability of integration	No lost order to fulfilment	Outbound queue with attempts, scheduled retry, dead-letter table, stranded-event alert, 10-min reconcile; idempotent payment finaliser; event sequence numbers reject stale updates (409).
Data integrity	Money and stock never drift	Integer pause; atomic stock ledger; settlement reconciliation against CCAvenue files; one-per-student Magic Box rule; qty caps on returns.
Observability	Detect 5xx and stuck jobs within 10 min	<code>instrumentation.ts</code> records unhandled errors to <code>storefront_events</code> ; support portal "Failures (24 h)"; cron logs under <code>/var/log/inventre-*.log</code> ; Prometheus exporters for node, Postgres, Redis, PgBouncer; Grafana. Partial Transitions (security posture lost or recovered, foreign commit, stranded ERP orders) e-mail <code>ALERT_EMAIL_TO</code> through the host mail relay (F-13); no central log shipping yet.
Maintainability	Type-clean, tested, reproducible builds	<code>tsc</code> zero errors gate; Vitest architecture invariants; ESLint 9 + Prettier; CI on GitHub Actions; deploy refuses dirty trees. Good 84 unit/domain/architecture tests plus a Playwright smoke and axe scan (<code>npm run smoke</code>) for storefront changes; largest page is now 1,150 lines (F-08).
Accessibility	WCAG 2.1 AA intent	Focus-visible ring, reduced-motion override, labelled controls, brand error and 404 boundaries, axe scan in the smoke run; open design decision on brand-orange contrast (F-15).
Compliance	Indian GST, PCI scope minimisation	Pincode-based in/out-state GST, HSN on lines, GSTR-1/3B data reports, e-invoice/e-way bill adapters scaffolded; card data never touches Inventre (hosted CCAvenue page).

10 Assumptions, constraints & risks

10.1 Assumptions

- Inventre's Postgres remains the storefront system of record; the Audit ERP is the fulfilment system of record. Neither replaces the other.
- Parents identify by mobile number; a phone may belong to several guardians and several students (family graph), never assumed one-to-one.
- Catalogue eligibility is decided server-side from the student's school and grade; the client only renders what it is given.
- The single production host has enough headroom for the season peak with caching; horizontal scaling is a future option, not a present need.

10.2 Constraints

- One host runs production, two test environments and two unrelated applications; port and container-name collisions are a real hazard (never `kill next-server` broadly).
- Production database reads are permitted for analysis; writes to production data are performed by the owner, not by tooling.
- Prod builds from the working tree; branch `main` may lag what is deployed.
- ERPNext is unreachable from the production host by firewall; only the Audit ERP is a live partner.

10.3 Risk register

#	Risk	Likelihood	Impact	Mitigation in place → recommended
R1	Deploy from a wrong checkout reverts production silently	Low (was High; happened twice 2026-09-10)	High	Compose renamed + guard in every other checkout; commit baked into every build and asserted after each deploy; monitor flags a foreign commit (F-01 closed)
R2	Single host failure takes down all environments	Low	High	Point-in-time archive + encrypted off-host mirror every 5 min + weekly restore test + rebuild runbook (F-10, F-11 closed)
R3	Cron-driven jobs stop silently (token rotated, container port change)	Low	Medium	One token in one file; stranded-order alert e-mails hourly; monitor e-mails on posture change (F-06, F-13 closed) → a per-job heartbeat metric remains a nice-to-have
R4	SMS balance exhaustion blocks all logins	Medium	High	<code>otp_logs</code> diagnostics → balance check in monitor + password fallback promoted in UI
R5	Audit ERP schema change breaks the poll or drain	Medium	Medium	Reconcile job + DLQ → contract tests against a recorded ERP fixture
R6	A new admin section ships without a permission gate	Low	Medium	All 39 sections gated + architecture test enforces it (F-02 closed) → keep the test in CI and deploy preflight
R7	Large client pages regress without tests	Low	Low–Medium	Pages split, error boundaries, <code>npm run smoke</code> against a local build before every storefront deploy (F-08 closed)

11 Findings & rectification register

Every drawback observed while producing this design is listed here with a concrete fix. Items are ordered by severity; closed items keep their row so the history stays readable. "Effort" is an engineering estimate: S ≤ 1 day, M ≤ 1 week, L multi-week.

ID	Severity	Finding (where)	Rectification	Effort
F-01	Fixed	Production built from the working tree; 16 checkouts shared one container name; two silent reverts on 2026-09-10	Closed 2026-09-23 (commit <code>934aab7</code>): <code>compose</code> file renamed + refusal guard in every other checkout (incl. the June backup copy); <code>next.config.mjs</code> bakes commit / branch / dirty / build time; <code>/api/version</code> reports them; <code>verify-deployment.sh</code> fails on a commit mismatch; <code>monitor-production.sh</code> logs a foreign commit. Remaining discipline: commit before deploying so <code>dirty</code> reads <code>false</code> .	—
F-02	Fixed	Earlier audit (2026-08-27) found 62 admin pages without a per-page guard	Verified closed on 2026-09-23: every section has a <code>SectionGate</code> layout and an architecture test enforces it. Remaining nit: <code>app/admin/page.tsx</code> sits outside <code>(protected)</code> ; it only redirects, but moving it inside removes the special case.	—
F-03	Fixed	Secrets on disk in several places: cron tokens in three <code>/etc/cron.d</code> files (two of them <code>.bak</code> copies), Postgres / PgBouncer / MinIO / SMS credentials hard-coded in <code>docker-compose.deploy.yml</code> and <code>deploy.sh</code> , DSNs in the MCB cron scripts, <code>.env.deploy.bak*</code> at the repo root	Closed 2026-09-23: one cron file sourcing <code>.env.deploy</code> ; <code>.bak</code> cron files deleted; <code>compose</code> and scripts use <code>\${VAR}</code> from <code>.env.deploy</code> ; <code>CRON_TOKEN</code> rotated, <code>CRON_SECRET</code> / <code>CRON_KEY</code> retired; env backups moved to <code>/root/.secrets-archive</code> (0700). Not rotated here: DB, Redis, JWT, gateway keys (tracked as F-04).	—
F-04	Owner action	Credential rotation after the June–July 2026 compromise	Done: JWT (dual-key, previous dropped same day), AUTH, support-view, Redis, MinIO, Grafana, OTP bypass, cron token, recovery-kit passphrase; Microsoft 365 app secret regenerated (<i>Inventre-prod</i> , old one deleted); portal and gift-card secrets pinned. Owner runs: <code>bash scripts/rotate-db-password.sh</code> (Postgres role password; ~1min interruption). Needs the other party: Audit ERP (webhook secret, poll password, support-view service token), CCAvenue working keys, Cloudflare R2 token, SMTP, SMS gateway, MyClassBoard API key, ERPNext clone credentials.	M
F-05	Fixed	Admin navigation defined twice (<code>AdminShell</code> and <code>lib/admin-nav.ts</code>) and drifted	Closed 2026-09-23: <code>lib/admin-nav.ts</code> is the only definition, keyed by page slug; the shell and the universal search both filter with <code>canSeePage</code> ; test <code>admin navigation > every sidebar item points at a registered admin page slug</code> .	—
F-06	Fixed	Cron endpoints <code>erp-stranded-alert</code> , <code>erp-webhook-drain</code> , <code>sync-items</code> existed but were not scheduled	Closed 2026-09-23: webhook drain every minute, stranded alert hourly (recipients from <code>ALERT_EMAIL_TO</code>); <code>sync-items</code> deliberately manual because the host cannot reach ERPNext. Verified live: both return 200 with counts.	—
F-07	Fixed	Cron jobs assumed a single app instance (no lock)	Closed 2026-09-23: <code>acquireCronLock</code> (Redis <code>SET NX PX</code> , compare-and-delete release) wraps all 12 jobs; a concurrent tick answers <code>{skipped: "locked"}</code> (verified with two simultaneous <code>erp-poll</code> calls).	—
F-08	Fixed	Two storefront pages exceeded 1,100 lines; tests were pure-function only	Closed 2026-09-23 (commit <code>cf0ca1d</code>): product page 1,337 → 1,150 lines (language pickers → <code>components/shop/pdp/LangPickers.tsx</code>); order page 1,176 → 718 (types, status pill, tracking card, stage bar → <code>components/shop/orders/*</code>); behaviour verified on a local build against the dev database with screenshots of shop, PDP, cart, orders, order detail. New <code>npm run smoke</code> (Playwright walk + axe-core WCAG A/AA per page) for every storefront change.	—
F-09	Fixed	Transactional SMS (order confirmed, exchange received) never sent while customer copy promised it	Closed 2026-09-23: no storefront copy promises an SMS (exchange status now says the page updates itself); skipped non-OTP sends are logged with the reason and return <code>skipped: "no-transactional-template"</code> . Enabling real transactional SMS needs approved DLT templates from the gateway (business decision).	—
F-10	Fixed	Single host; no failover	Closed as designed 2026-09-23: <code>docs/runbooks/host-rebuild.md</code> gives the full rebuild path from a snapshot with a measured budget (~65–75 min, plan 2 h). A second host or streaming replica remains a cost decision for the owner.	—

ID	Severity	Finding (where)	Rectification	Effo
F-11	Fixed	Backups stayed on the same host; the R2 upload had been failing since 2026-09-15	Closed 2026-09-23. Database: continuous WAL streaming (systemd <code>inventre-wal-stream</code>) + full backup 00:00 IST + logical dump 00:10 → recovery to any second. Code: git bundle and GitHub push (<code>Snith0801/Inventre-Website</code> , deploy key) on every commit, after every deploy and at 00:20; uncommitted edits packed every 5 min. Configuration: <code>.env.deploy</code> re-encrypted and mirrored within 5 min of any change (and inside the 6-hourly snapshot). Off-host: rclone every 5 min to Microsoft 365, SharePoint <i>Vendor Management Files AUDIT > Documents > Inventre Backups > prod</i> , encrypted on the server first (names and contents). Recovery kit: the same folder's <code>RESTORE/</code> holds <code>bootstrap-new-server.sh</code> , the restore engine, a README and <code>recovery-kit.env.gpg</code> (all secrets, one passphrase) — a new server needs the folder plus one passphrase. Proof: first mirror verified by download-and-compare; weekly restore test with e-mail; failure and staleness e-mails. To do	—
F-12	Fixed	<code>next/image</code> configured but unused; <code>lib/cdn-image.ts</code> dead	Closed 2026-09-23: dead helper deleted; the <code>images</code> config carries a note recording the decision (plain <code></code> from R2).	—
F-13	Fixed	No alert routing; cron logs only on disk	Closed 2026-09-23: the 10-minute monitor e-mails <code>ALERT_EMAIL_T0</code> through the host mail relay on every transition (posture lost, recovered, foreign commit); the stranded-order alert is scheduled hourly.	—
F-14	Fixed	Stale plans described an unbuilt stack; stale ESLint comment; scaffold and empty <code>lib/*</code> folders	Closed 2026-09-23: plans moved to <code>docs/history/</code> ; README points at this HLD/LLD; comment fixed; scaffold READMEs and empty folders removed.	—
F-15	Fixed	design note No error boundaries; no a11y tooling	Closed in code 2026-09-23: error / not-found boundaries; axe scan in <code>npm run smoke</code> ; unlabelled school, grade and sort selects fixed; order summary list made valid; decorative 404 numeral hidden. Design decision for the owner: the only remaining serious finding is white text on brand orange <code>#E47127</code> (3.13:1; WCAG AA wants 4.5:1 for body-size text) and orange eyebrow text on cream (2.93:1). Fix is a palette change (for example <code>brand-600</code> for filled buttons, <code>brand-700</code> for eyebrows), not a code defect.	—
F-16	Fixed	Storefront tabs kept stale JavaScript after a deploy	Closed 2026-09-23: <code>VersionWatcher</code> in careful mode on the shop layout — idle tabs reload, a parent who has typed anything gets a prompt instead.	—
F-17	Fixed	Cron endpoints authenticated five different ways	Closed 2026-09-23: <code>server/cron-auth.ts</code> <code>requireCron()</code> used by all 12 routes; architecture test <code>scheduled jobs > every /api/cron</code> route authenticates through <code>requireCron</code> enforces it. Verified live: valid token 200, old header style 401, wrong token 401.	—
F-18	Fixed	Two validation helpers with different 400 shapes; unused <code>apiHandler</code>	Closed 2026-09-23: one payload <code>{error, details[], issues[]}</code> ; <code>parseBody</code> is an alias of <code>parseJson</code> ; dead wrapper removed.	—
F-19	Fixed	Duplicate migration prefixes, nested <code>db/migrations/migrations/</code> copy, ERP-owned <code>erp.*</code> tables absent from a rebuild	Closed 2026-09-23 for what is safe: nested copy deleted; architecture test forbids any <i>new</i> duplicate prefix (the ten historical ones stay — renaming a file would re-run it on every host). ERP-owned mirror tables are documented in LLD §5.5 as created by the ERP; an empty bootstrap migration is deliberately not added because their columns are owned by the other system.	—
F-20	Fixed	Business spreadsheets, PDFs and a build artefact at the repository root	Closed 2026-09-23: files moved to <code>/root/deliverables</code> , root <code>*.xlsx / *.pdf / *.docx</code> ignored, tracked <code>server.js</code> build artefact removed.	—
F-22	Fixed	docker-compose v1 crashes on <code>--force-recreate</code> of an existing container (Redis outage 2026-09-23)	Closed 2026-09-23: the compose v2 plugin (already installed, v5.1.4) is used by <code>deploy.sh</code> , the DB-rotation and the restore scripts; the stop-then-up path is kept as belt and braces.	—
F-23	Fixed	<code>minio-init</code> depended on <code>minio</code> being <i>healthy</i> while minio's healthcheck is disabled, so any compose <code>up</code> that included it failed	Closed 2026-09-23 (commit <code>96eb4c9</code>): plain dependency.	—

ID	Severity	Finding (where)	Rectification	Effo
F-21	Fixed	Found during F-17 verification: /api/cron/cleanup-carts had returned 500 on every tick (a JS Date passed through a raw sql fragment), so expired carts were never purged	Closed 2026-09-23 (commit bff65e6): typed Drizzle operators; first successful run deleted 7,719 stale carts.	—

12 Glossary & references

12.1 Glossary

Audit ERP	In-house fulfilment and call-centre ERP (FastAPI + Postgres) on 217.216.58.218, referred to as "audit" in code and logs.
Magic Box	A configurable bundle (uniform + books + accessories) sold as one product; one per student; components chosen in the configurator and stored as <code>bundle_selections</code> .
Bookkit	A fixed book bundle per school and grade; Grade 11-12 kits carry Mandate × Core × Elective subject variants and 2nd/3rd language variants.
Ground Stock	Physically counted stock per SKU maintained in the Audit ERP; bridged into Inventre bins every 5 minutes.
RTN / MIS / CON	Exchange request, missing-item claim, concern ticket. Numbers minted by the Audit ERP (<code>RTN-M-</code> , <code>MIS-M-</code>) or Inventre (<code>RTN-YYYY</code> , <code>CON-</code>).
MCB	MyClassBoard, the school management system that holds student rosters, fee schedules and receipts for 13 branches.
Packing state	Per-item fulfilment state from the ERP used to gate exchange/missing buttons and to badge order lines.
Support view	Read-only impersonation session an Audit agent uses to see a parent's storefront.
IELP / IESP	Parcel codes: LARGE parcel (bookkits) and SMALL parcel (uniforms); one order may split into both.

12.2 References

Document	Location
Low-Level Design (companion)	Inventre — LLD v1.0 (same date)
Developer setup	<code>README.md</code>
Design system spec	<code>docs/design/DESIGN_SYSTEM.md</code>
ERP staging integration notes	<code>docs/architecture/INTEGRATION.md</code>
Exchange & missing flow	<code>docs/flows/EXCHANGE_AND_MISSING_FLOW.md</code> , <code>EXCHANGE_MISSING_TEST_SCENARIOS.md</code>
Runbooks	<code>docs/runbooks/ground-stock-bridge.md</code> , <code>single-deploy-source.md</code> , <code>deploy/README.md</code> , <code>deploy/PRODUCTION-MIGRATION.md</code>
Refactor baseline audit	<code>docs/architecture/REFACTOR-AUDIT-2026-09-10.md</code> , <code>docs/pre-refactor-baseline.md</code>
Superseded plans (historical only)	<code>docs/history/PLAN.md</code> , <code>ERP_INTEGRATION_PLAN.md</code> , <code>ERP_REPLACEMENT_PLAN.md</code> , <code>OPTION_B_PLAN.md</code>
Host rebuild runbook	<code>docs/runbooks/host-rebuild.md</code>
Security	<code>docs/security/SAST-20260909</code> , <code>scripts/monitor-production.sh</code>
Monitoring stack	<code>deploy/monitoring/README.md</code>